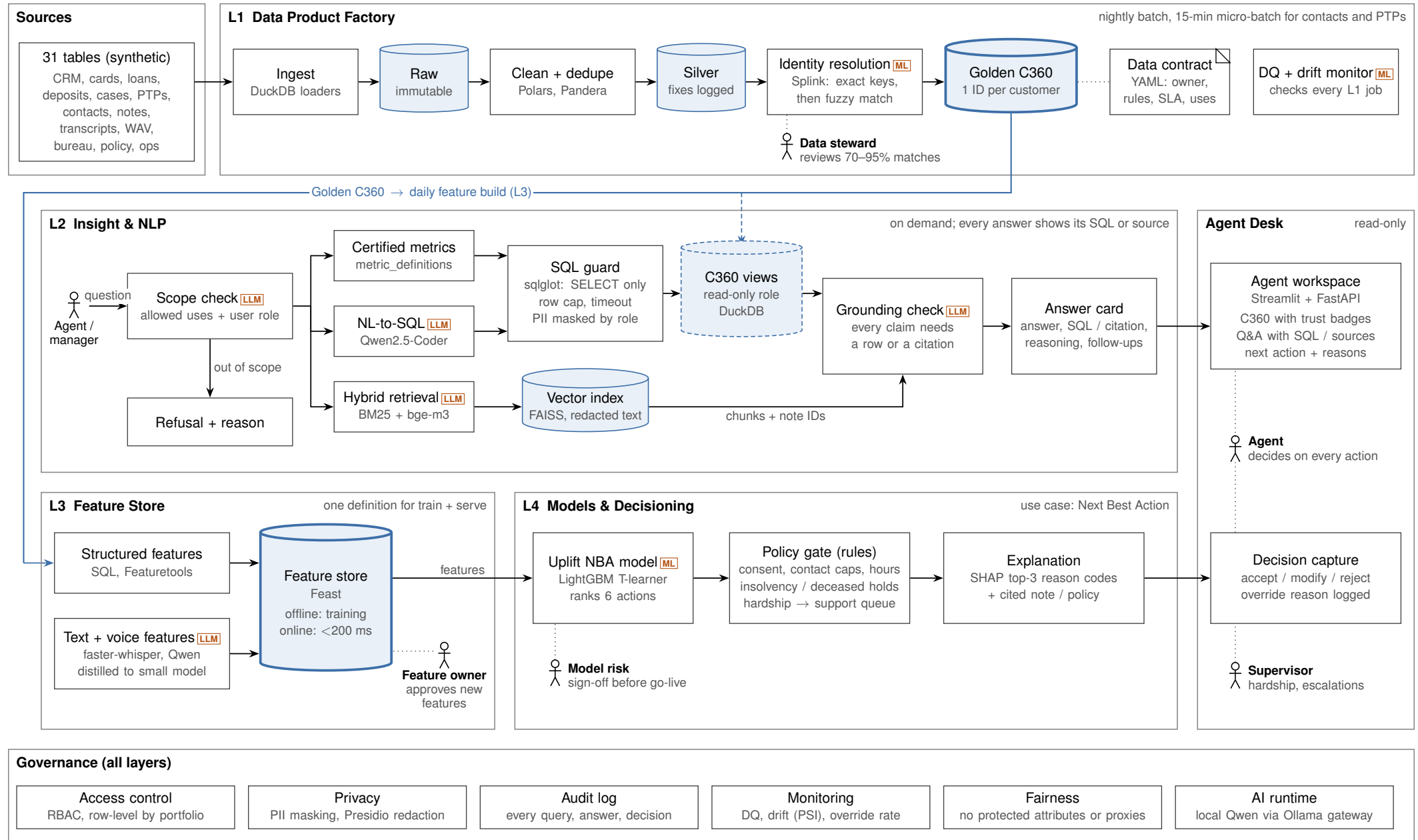


Collections 360 | System Architecture

Kondameedi Srujan Raj, D Varshith Reddy, G Swachatha, T Sreenidhi

Team: Straw Hats | 1. Architecture diagram

CIBC Collections Hackathon, IIIT Hyderabad | Use case: Next Best Action



Legend data store data product (has a contract) read-only view AI component human review point data flow read-only access association

2. Design document

2.1 Summary

Problem: data sits in separate CRM, card, loan, deposit and collections systems with different keys, so agents lack context.

Use case: Next Best Action, recommending the action most likely to cure each open case, with plain-English reasons.

Users: collections agents and team leads during contact; strategy managers asking questions in plain English.

2.2 Business value

Metric	Definition	Role	Source
30-day cure rate	cases back to current within 30 days of entering a bucket ÷ cases entering	primary	cure_flag, days_to_cure
Roll rate	cases moving to the next bucket in the month ÷ cases in the bucket	primary (mirror)	roll_forward_count
Promise-kept rate	promises paid in full by due date + grace ÷ promises made	leading	promises_to_pay
Right-party contact rate	right-party contacts ÷ attempts	leading	rpc_flag
Contacts per cure, cost	attempts and contact_cost_cad per cured case	efficiency	contact_history
Guardrails	complaints, contact-cap breaches (must stay 0), share of hardship cases offered support, override rate by segment	must not worsen	cases, contacts

How we measure it. The baseline is the current strategy (`strategy_code`) and the incumbent NBA (`nba_recommendation_last`). The bank already randomises treatments (`test_cell`), so we can (1) back-test offline on past randomised cells, using a Qini curve and cure rate by uplift decile, and (2) run a live pilot where NBA serves a random 10–20% of new cases for 8–12 weeks. We report the cure-rate difference against control with a 95% confidence interval, size the pilot from the baseline rate (minimum detectable effect TBC), and stop early if any guardrail is breached.

Value. Δ cure rate $\times$ balance entering collections $\times$ loss rate on rolled balances, plus agent hours saved on self-curing customers who no longer get a call (attempts avoided $\times$ handle time). Figures are filled in from the data (TBC).

2.3 Layer 1: building the C360 and matching customer IDs

Stage	What happens	Tools	Output
Raw	files land unchanged (CSV, Parquet by year, JSON, WAV); schema, row count and file hash recorded	DuckDB, Parquet	<code>raw.*</code> , immutable
Silver	DOBs to ISO dates, phones to E.164, postal codes to A1A 1A1, timestamps to UTC with local zone; duplicates removed by <code>record_hash</code> ; current CDC rows only; latest snapshot per account; codes decoded via <code>code_lookups</code> ; every fix logged	Polars, Pandera	<code>silver.*</code> , fix log (rule, table, rows)
Identity	CRM clean-up, exact links, probabilistic matching, steward queue	Splink (DuckDB backend)	<code>id_xref</code>
Gold	survivorship rules, C360 tables, field-level lineage and trust	DuckDB SQL	<code>c360_customer</code> , <code>_account</code> , <code>_case</code>
Publish	contract checks, DQ report, versioned nightly snapshot	Pandera, YAML contract	DQ report, C360 version

Matching customer IDs. Each product system has its own key, so we match *source keys* (system, `src_customer_ref`), not rows:

- CRM clean-up:** merge duplicate CRM records via `duplicate_of_crm_id` and `national_id_hash`.
- Exact links:** `account_monthly_snapshot`, `salary_credit_history`, `contact_history`, `promises_to_pay` and `agent_notes` carry both an account ID and `crm_customer_id`. Bureau pulls likely join on the deposits CIF (`bank_subject_ref`, TBC). Stale `coll_customer_ref` values are resolved through `id_xref`.
- Probabilistic links (Splink, Fellegi–Sunter):** name with Jaro–Winkler and a nickname list (`preferred_name`), DOB exact or day/month swapped, phone on last 10 digits, postal code exact or FSA, email lower-cased. Blocking on FSA + birth year and on phone. Score ≥ 0.95 merges, 0.70–0.95 goes to a data steward, below 0.70 stays separate.
- Accuracy:** exact links are known-true pairs, so we report precision and recall of the matcher (target precision $\geq 98\%$, TBC) and the per-field match weights behind each decision.

Survivorship. Consent: most restrictive value. Hardship, vulnerability, insolvency, deceased, cease-contact: true if any source is true. Phone, email, address: most recently verified (`email_verified_flag`, `last_profile_update_ts`). Balances and DPD: system of record. Joint accounts and co-borrowers (`joint_holder_ref`, `co_borrower_ref`) are links with a role, and one borrower's details never appear on the other's record.

Quality and monitoring. Checks cover schema, completeness, validity, uniqueness, referential integrity and freshness. A monitor compares volumes, null rates and distributions with the previous run and flags schema drift (e.g. `channel` VS `channel_v2`); it is validated against the outages in `ops_events`. Every C360 field carries source, refresh time and DQ status, shown to agents as a trust badge.

Example (illustrative keys). CRM record RYM-1142, card customer C-77310, loan borrower L-20981 and deposit CIF 0045512 belong to one person. The loan and deposit keys link exactly through `account_monthly_snapshot`; the card key, typed “RYAN J MITCHELL” with DOB 14/03/1988, matches by Splink at 0.97. All four get one `golden_id`, and `id_xref` stores the method, score and field weights for each link.

2.4 Layer 2: from question to answer

#	Step	What it does
1	Scope check	rules first (protected attributes, write verbs, data outside the user's role), then a local LLM classifier for reworded questions; both read the contract's allowed uses
2	Understand	extracts metric, product, segment, period and filters as editable tokens shown to the user
3	Route	certified metric (<code>metric_definitions</code> synonyms → approved formula); NL-to-SQL (Qwen2.5-Coder, local, schema + metric context, few-shot from benchmarks); or hybrid RAG (BM25 + bge-m3, reranked) over <code>agent_notes</code> , <code>call_transcripts</code> , <code>reference_documents</code>
4	SQL guard / scoped retrieval	see below; retrieval is filtered by role, case and current policy version (<code>effective_date</code> , <code>superseded_by</code>)
5	Grounding check	every sentence must cite a returned row or chunk ID; unsupported sentences are removed
6	Answer card	answer, tokens, SQL or citations (note ID, transcript turn, policy section), one-line reasoning, certified/exploratory badge, two follow-ups

SQL guard (five independent controls). (1) read-only DuckDB connection; (2) sqlglot AST check: exactly one SELECT, no DDL/DML, no multiple statements; (3) table and column allowlist from the contract; (4) automatic LIMIT 1,000 and 10 s timeout; (5) role-based views with PII masked. A query that fails any control is not run and the reason is logged.

Refusal category	Example	Response
Protected attribute	"Which customers are women?"	declined, reason given
Write or action	"Delete this note", "move the case to legal"	declined; offers a recommendation for a person
Outside role	"Show another agent's customers"	declined by access rules
Outside the data	"What's the share price today?"	declined, suggests an in-scope question
Unsafe request	"Write a threatening message"	declined, logged
Injected instruction	a note saying "ignore your rules"	retrieved text treated as data only

Evaluation. On `benchmark_questions`: execution accuracy within `tolerance` (target $\geq 85\%$, TBC), refusal accuracy on `should_refuse` rows (target 100%), zero non-SELECT statements executed, p95 latency under 8 s. Plus our own injection and DROP/DELETE test set.

2.5 Layer 3 (Feature Store)

Feature	Type	Built from	Refresh	Why
<code>dpd_current</code> , <code>dpd_slope_3m</code>	SQL, stat	<code>dpd</code> , <code>dpd_m01-m03</code>	nightly	level and direction of arrears
<code>ptp_broken_count_90d</code>	SQL	<code>promises_to_pay.ptp_status</code>	15 min	best predictor of a broken promise
<code>ptp_payday_gap_days</code>	SQL	<code>ptp_due_date</code> — <code>expected_next_payroll_date</code>	15 min	promises due before payday break
<code>payroll_delay_days</code> , <code>salary_change_3m</code>	SQL, stat	<code>deposits</code> , <code>salary_credit_history</code>	nightly	income shock
<code>nsf_count_3m</code> , <code>cash_flow_slope_6m</code>	SQL, stat	<code>nsf_count_m*</code> , <code>net_cash_flow_m*</code>	nightly	ability to pay
<code>utilisation_trend_6m</code>	stat	<code>utilisation_m01-m06</code>	nightly	borrowing to cover gaps
<code>bureau_delta_90d</code> , <code>off_us_delinquent</code>	SQL	<code>external</code>	monthly	stress at other lenders
<code>answer_rate_30d</code> , <code>best_time_band</code>	SQL	<code>contact_history</code>	15 min	reachability, channel and timing
<code>contacts_last_7d</code>	SQL	<code>contact_history</code>	15 min	contact-cap rule and fatigue
<code>hardship_signal</code> , <code>delay_reason</code>	LLM	<code>notes</code> , <code>transcripts</code>	nightly	reason for non-payment
<code>ptp_intent_strength</code> , <code>call_sentiment</code>	LLM	<code>transcripts</code> , <code>turn.sentiment_score</code>	nightly	credibility of the promise
<code>talk_ratio</code> , <code>silence_ratio</code>	voice	<code>transcripts</code> ; WAV via faster-whisper	nightly	engagement on the call

How features are generated and selected. SQL aggregates and statistical transforms (trend, change, volatility, recency, ratios) come first. Featuretools then generates candidates from recipes (column × aggregation × window of 30/90/180 days). A candidate is kept only if it passes five tests: predictive (IV/WoE, mutual information), not redundant (correlation < 0.9), stable (PSI < 0.2 across months), not a proxy for a protected attribute, and computable at decision time. Outcome columns (`cure_flag`, `dpd_at_day_*`, `nba_accepted_flag`) are labels, never inputs. A new feature suggested by the LLM needs a feature owner's approval.

Text and voice. Qwen labels a stratified sample of about 20,000 notes and transcripts with a versioned prompt; we hand-label 300 to measure accuracy (target $F1 \geq 0.8$, TBC). A small multilingual classifier (EN/FR) is trained on those labels and scores all 1.5M notes on CPU. WAV files without transcripts go through faster-whisper. Voice pitch, energy and accent are not used. Each LLM feature stores its prompt, version and accuracy.

Same values for training and live scoring. One Feast feature view per entity (customer, account, case) defines each feature once. Training uses point-in-time joins at the decision timestamp from the offline store (Parquet). The online store is materialised from the same views nightly and every 15 minutes for contact and promise features, serving in under 200 ms. A daily parity test compares offline and online values for sampled keys (tolerance 0.1%), and every score stores the feature-view version it used.

Catalog card. Every feature is registered with name, entity, type, plain-English meaning, recipe (SQL, or prompt ID and version), source columns, refresh, owner, version, the reason we keep it and, for LLM features, accuracy against human labels. This is what the agent screen and model card link to.

2.6 Layer 4 (Models and Decisioning)

Use case and actions. Next Best Action for each open case, choosing among: (A1) no contact, likely self-cure; (A2) SMS or email with a pay link; (A3) call in the best time band; (A4) short-term payment plan; (A5) hardship programme referral; (A6) escalation, recommendation only.

Model. T-learner uplift: one LightGBM classifier per action estimates $p_a = P(\text{cure in 30 days} \mid \text{features}, a)$, trained on cases in randomised `test_cells`, with time-based splits. The recommended action maximises

$$\text{gain}_a = (p_a - p_{A1}) \times \text{balance at risk} - \text{cost}_a$$

subject to channel capacity (`channel_capacity`) and agent capacity (`agent_shifts`). Validation: Qini/AUUC on held-out cells, calibration, uplift by decile and by segment. If randomised history is too thin, we fall back to per-action propensity models and report that.

Policy gate rule (after the model)	Data	Effect
Insolvency or deceased	<code>bankruptcy_flag</code> , <code>consumer_proposal_flag</code> , <code>insolvency_hold_flag</code> , <code>deceased_flag</code>	no collection contact; route to specialist
Cease contact, open dispute Representative	<code>cease_communication_flag</code> , <code>dispute_flag</code> , <code>credit_counselling_agency_flag</code> , <code>third_party_rep_flag</code>	no contact until resolved contact the representative only
Consent, caps, hours	<code>consent_*</code> , <code>contacts_last_7d</code> vs <code>contact_cap_7d</code> , permitted hours, time zone	channel or time removed
Hardship Vulnerability	<code>hardship_flag</code> , <code>hardship_programs_eligibility</code> , <code>vulnerability_flag</code> , <code>vulnerable_customer_certified</code>	supportive action only; programme offered certified agent; supervisor queue

How each decision is explained. SHAP values for the chosen action become the top three reason codes in fixed templates, each linked to its evidence, plus a confidence level and the policy checks applied. Example: “Offer a 3-month payment plan by phone, 6–8 PM. Salary 12 days late; 2 promises broken in 90 days; note N-48213 mentions reduced shifts (hardship). Checks passed: consent, contact cap. Confidence: medium.” Wording is deterministic; the LLM does not write explanations.

Where a human reviews. The agent accepts, edits or rejects every recommendation and gives a reason code when overriding. Hardship, vulnerable, escalated and low-confidence cases go to a supervisor queue first. Each model version needs model-risk sign-off. Override rates and reasons are monitored by segment and fed into retraining.

Reuse. Layers 1–3 serve the other use cases too: channel choice and vulnerable-customer routing are already inside NBA; **After go-live.** We track acceptance and override rates by reason and segment, cure rate against the control cell, calibration, and PSI drift on the top features. A kill switch returns all cases to the current strategy if a guardrail is breached, and every new model version starts as a challenger on a small random share of cases.

agent assist, post-call summaries (draft note edited by the agent) and QA scoring against `qa_checklist` reuse the same data and features.

2.7 AI usage

Where	What we use	Why AI helps	Guardrail
Customer matching (L1)	Splink probabilistic model	same person typed differently per system	thresholds, steward review, accuracy on known links
Pipeline monitor (L1)	statistical anomaly detection	silent data breaks	tested on past outages; alerts only
Source mapping (L1)	LLM on column metadata	31 tables, ~1,400 columns	no customer rows sent; engineer approves
Scope check (L2)	rules + LLM classifier	reworded out-of-scope questions	rules decide first; refusals logged
NL-to-SQL (L2)	Qwen2.5-Coder, local	new questions without new reports	SQL guard; certified metrics first
Notes and policy Q&A (L2)	bge-m3 + BM25, LLM summary	answers live in free text	citations required; grounding check
Text and voice features (L3)	LLM teacher, small student, faster-whisper	hardship reasons only appear in words	human-label accuracy; versioned prompts
NBA model (L4)	LightGBM uplift + SHAP	interactions of income, behaviour, channel	policy gate, reason codes, sign-off

No AI in: the policy gate, explanation wording, SQL execution, access control, or the final decision. All models run locally behind one gateway (Ollama) that redacts PII with Presidio and logs prompt, model and version; no customer data goes to an external API.

2.8 Scalability

Component	Batch or real time	Volume, frequency, latency
Ingest, silver, matching, C360	batch + micro-batch	nightly full rebuild (~1M customers, ~1.84M accounts, history Parquet by year); cases, contacts, promises every 15 min
Feature store	batch + micro-batch	offline nightly; online refresh every 15 min for contact and promise features
Text features, search index	batch, incremental	new notes and transcripts only; distilled model on CPU; LLM only for samples and new prompt versions
Model training	batch	weekly or monthly; champion/challenger before promotion
NBA scoring	batch + real time	all open cases overnight; re-score in <200 ms when a case is opened or a contact is logged
Plain-English Q&A	real time	on demand; 10 s timeout, row cap, cached certified metrics; p95 < 8 s
Monitoring and audit	batch + streaming log	daily DQ, drift and fairness reports; audit events written as they happen

Production path: DuckDB → Snowflake or Databricks; micro-batch files → Kafka; Feast online → Redis; FAISS → managed vector DB; Ollama → vLLM on bank GPUs; Streamlit → internal web app. Components are containerised and stateless, so scoring and Q&A scale horizontally.

2.9 Governance

Role	Can see	Can do
Agent	own queue; PII partly masked; vulnerability type hidden unless certified	accept, edit, reject recommendations
Team lead	own team's cases and overrides	approve overrides, escalations
Supervisor	hardship, vulnerable, escalated queue	approve or change action before contact
Strategy manager	aggregates only, no free text, no PII	Q&A, reports
Data steward	match queue with evidence	approve or reject links (0.70–0.95)
Model risk	audit log, model cards, fairness reports	sign off model versions

Human review. Five points: H1 data steward (matches), H2 feature owner (new features), H3 agent (every action), H4 supervisor (hardship, vulnerability, escalation, low confidence), H5 model risk (each model version). Nothing that affects a customer happens without a person approving it; the AI never writes to source or golden data.

No protected attributes. A schema test keeps `gender_code`, `marital_status`, `citizenship_status`, `household_size`, `dependants_count`, `newcomer_program_flag`, `newcomer_arrival_date`, `ccb_deposit_flag` and `customer_accent_region` out of every model and rule. Likely proxies are also dropped: `age`, `age_band`, `cpp_oas_deposit_flag`, `voice_pitch` and `energy`, `FSA`, `fsa_income_index`, `majority_language`. Protected attributes live only in a fairness view used to compare outcomes and override rates by group. Incumbent scores with `uses_protected_attributes = true` are excluded.

Hardship flags. Raised from numbers (payroll delay, salary drop, EI deposits, NSF events, `garnishment_flag`, existing `hardship_flag`) or text (`hardship_mentioned_flag`, `vulnerability_disclosed_flag`, LLM `hardship_signal`). The flag only protects: supportive actions, a `hardship_programs` check, a certified agent and the supervisor queue; it never increases contact. Accessibility needs only adapt the channel.

Monitoring, audit, security. Every query, SQL, answer, recommendation and override is logged with user, time and model version. Daily reports cover data quality, drift, accuracy and outcome gaps between groups. Only redacted text is indexed; retention dates (`retention_until`) purge the index. Model governance follows OSFI Guideline E-23 principles; data handling follows PIPEDA.

2.10 Build plan (36 hours, 3 Oct 09:00 to 4 Oct 21:00)

Hours	Srujan: Layer 1	Varshith: Layers 3–4	Swachatha: Layer 2	Sreenidhi: desk, governance
0–6	load, profile all 31 tables	feature list on real columns	schema notes, metric layer	UI skeleton, contract v1
6–14	silver, matching, C360, DQ report	SQL features, LLM labels	NL-to-SQL, SQL guard	role views, audit log
14–24	steward queue, match accuracy	Feast store, uplift model	RAG index, grounding	policy gate, reason templates
24–32	15-min updates	SHAP, fairness checks	benchmark run, refusal tests	agent screens end to end
32–36	all: integration test, 5-minute demo video, 10-slide deck, README with one-command rebuild			

Risks and fallbacks. Thin randomised history → per-action propensity models. Slow LLM labelling → smaller sample, keyword baseline. Weak NL-to-SQL → more certified metrics and few-shot examples. Time pressure → drop voice, then use a precomputed cache instead of the online store. Checkpoints every 6 hours with a working demo.

Demo (5 min). Raw files to golden record with the match report and one steward decision; three benchmark questions with SQL and citations, plus one refusal; an agent opens a hardship case, reads the recommendation and reasons, overrides with a reason code; the audit log and fairness report for that session.

3. Draft data contract: Golden C360 customer record (`c360_customer`), v0.1

Owner	Business owner: Head of Collections Strategy (name TBC). Data product owner: Collections Data team. Steward: Collections data steward (match queue). Technical owner: data engineering.
Consumers	Layer 2 Q&A, Layer 3 feature store, Agent Desk, collections reporting.
Grain and key	One row per customer, primary key <code>golden_id</code> . Linked: <code>id_xref</code> , <code>c360_account</code> , <code>c360_case</code> .
Classification	Confidential, contains PII. Served through role-based masked views.

Schema

Field	Type	Source	Class	Note
<code>golden_id</code>	string, PK	<code>id_xref</code>	internal	stable across rebuilds
<code>source_keys</code> , <code>match_confidence</code>	array, dec(4,3)	<code>id_xref</code> , Splink	internal	lowest link score
<code>display_name</code> , <code>primary_phone_e164</code> , <code>email</code>	string	customers (survivorship)	PII	masked by role
<code>segment</code> , <code>province_code</code> , <code>time_zone</code>	enum	customers	quasi-ID	province sets contact rules
<code>preferred_language</code>	enum	customers	quasi-ID	service language only
<code>tenure_months</code> , <code>product_count</code>	int	customers, derived	internal	
<code>total_credit_exposure</code>	dec(14,2)	card and loan accounts	confidential	CAD
<code>max_dpd_current</code> , <code>worst_dpd_12m</code>	int	card and loan accounts	confidential	max across accounts
<code>active_case_id</code> , <code>current_bucket</code>	string, enum	<code>collections_cases</code>	confidential	null if no open case
<code>open_ptp_flag</code> , <code>ptp_broken_count_90d</code>	bool, int	<code>promises_to_pay</code>	confidential	
<code>last_payroll_date</code> , <code>payroll_delay_days</code>	date, int	<code>deposit_accounts</code>	confidential	
<code>bureau_risk_score</code> , <code>bureau_delta_90d</code>	int	external	confidential	latest pull
<code>consent_call sms email push</code>	bool	customers	internal	most restrictive value
<code>cease_communication_flag</code>	bool	customers, cases	internal	true if any source true
<code>permitted_call_start</code> , <code>_end</code>	time	customers	internal	customer local time
<code>hardship_flag</code>	bool	cases, accounts, text	sensitive	protective use only
<code>vulnerability_flag</code>	bool	customers	sensitive	type not included
<code>insolvency_flag</code> , <code>deceased_flag</code>	bool	customers, external, cases	sensitive	hard stops
<code>third_party_rep_flag</code>	bool	customers, cases	internal	contact representative
<code>field_trust</code>	map	pipeline	internal	source, refreshed_at, dq_status
<code>as_of_ts</code> , <code>c360_version</code>	ts, string	pipeline	internal	

Quality rules

#	Rule (dimension)	Threshold	If it fails
Q1	<code>golden_id</code> unique; each source key maps to exactly one <code>golden_id</code> (uniqueness)	100%	don't publish
Q2	automatic links have <code>match_confidence</code> ≥ 0.95 ; 0.70–0.95 only after steward approval (accuracy)	100%	link held back
Q3	consent fields never null; missing stored as false, i.e. no contact (completeness)	100%	no contact, alert
Q4	<code>max_dpd_current</code> ≥ 0 and consistent with <code>current_bucket</code> (validity)	99.5% (TBC)	flag in <code>field_trust</code>
Q5	every open case resolves to one <code>golden_id</code> (referential integrity)	99.9% (TBC)	case to steward
Q6	<code>as_of_ts</code> within 24 h; 15-minute fields within 30 min (freshness)	100%	stale badge on screen
Q7	no protected-attribute or proxy column present (schema test)	0 columns	don't publish

Use, refresh and change

Allowed uses	collections treatment decisions (Next Best Action); the agent screen; collections reporting and plain-English Q&A; training and monitoring of collections models.
Not allowed	marketing or cross-sell; credit limit or pricing decisions; use outside Collections without owner approval; using hardship or vulnerability flags to increase pressure; write-back by any AI component; any use of protected attributes.
Refresh frequency	full rebuild nightly, ready by 06:00 ET (TBC); case, contact and promise fields every 15 minutes.
SLA	99% availability in collection hours (TBC); freshness as in Q6; DQ report published with every run.
Lineage	each field records source table and column, refresh time and the checks it passed.
Retention	follows source retention (<code>retention_until</code>); nightly snapshots kept for audit (period TBC).
Changes	new fields in a minor version; breaking changes need a new major version, owner sign-off and notice to consumers (period TBC).
Machine-readable copy	kept as YAML next to the data; Layer 2 reads allowed uses and column classes from it at runtime.